

Assignment 10: Data Scraping

Josh Salzberg

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on data scraping.

Directions

1. Rename this file `<FirstLast>_A10_DataScraping.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up

1. Set up your session:
 - Load the packages `tidyverse`, `rvest`, and any others you end up using.
 - Check your working directory

```
#1
library(tidyverse); library(rvest); library(here)
here()
```

```
## [1] "/home/guest/project/EDE_Fall2025"
```

2. We will be scraping data from the NC DEQs Local Water Supply Planning website, specifically the Durham’s 2024 Municipal Local Water Supply Plan (LWSP):
 - Navigate to <https://www.ncwater.org/WUDC/app/LWSP/search.php>
 - Scroll down and select the LWSP link next to Durham Municipality.
 - Note the web address: <https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=03-32-010&year=2024>

Indicate this website as the as the URL to be scraped. (In other words, read the contents into an `rvest` webpage object.)

```
#2
MyURL <- 'https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=03-32-010&year=2024'
MySite <- read_html(MyURL)
MySite
```

```
## {html_document}
## <html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
## [1] <head>\n<title>DWR :: Local Water Supply Planning</title>\n<meta http-equ ...
## [2] <body id="plan">\r\n<!--<div id="division-header">\r\n<a name="top" href= ...
```

3. The data we want to collect are listed below:

- From the “1. System Information” section:
- Water system name
- PWSID
- Ownership
- From the “3. Water Supply Sources” section:
- Maximum Day Use (MGD) - for each month

In the code chunk below scrape these values, assigning them to four separate variables.

HINT: The first value should be “Durham”, the second “03-32-010”, the third “Municipality”, and the last should be a vector of 12 numeric values (represented as strings)“.

```
#3
SystemName <- MySite %>%
  html_nodes("div+ table tr:nth-child(1) td:nth-child(2)") %>%
  html_text()
SystemName
```

```
## [1] "Durham"
```

```
PWSID <- MySite %>%
  html_nodes("td tr:nth-child(1) td:nth-child(5) ") %>%
  html_text()
PWSID
```

```
## [1] "03-32-010"
```

```
Ownership <- MySite %>%
  html_nodes("div+ table tr:nth-child(2) td:nth-child(4)") %>%
  html_text()
Ownership
```

```
## [1] "Municipality"
```

```
MaxDailyUse <- MySite %>%
  html_nodes("th~ td+ td") %>%
  html_text()
MaxDailyUse <- as.factor(MaxDailyUse)
```

4. Convert your scraped data into a dataframe. This dataframe should have a column for each of the 4 variables scraped and a row for the month corresponding to the withdrawal data. Also add a Date column that includes your month and year in data format. (Feel free to add a Year column too, if you wish.)

TIP: Use `rep()` to repeat a value when creating a dataframe.

NOTE: It's likely you won't be able to scrape the monthly withdrawal data in chronological order. You can overcome this by creating a month column manually assigning values in the order the data are scraped: "Jan", "May", "Sept", "Feb", etc...

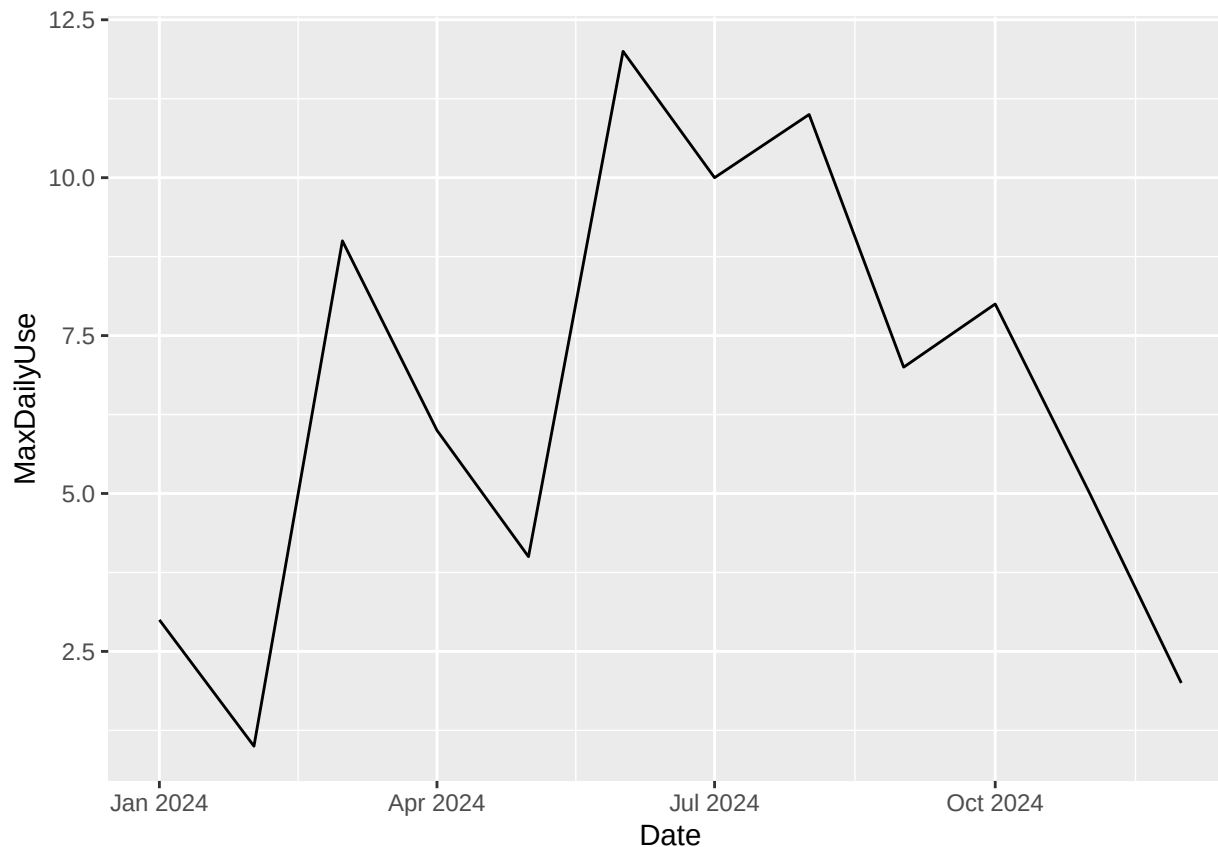
5. Create a line plot of the maximum daily withdrawals across the months for 2024, making sure, the months are presented in proper sequence.

```
#4
MyFrame <- data.frame(Date = my(c("1, 2024", "2, 2024", "3, 2024", "4, 2024",
                                "5, 2024", "6, 2024", "7, 2024", "8, 2024",
                                "9, 2024", "10, 2024", "11, 2024", "12, 2024")),
                      Ownership = rep(Ownership, 12),
                      PWSID = rep(PWSID, 12),
                      System = rep(SystemName, 12),
                      MaxDailyUse = as.numeric(c(MaxDailyUse[[1]], MaxDailyUse[[4]], MaxDailyUse[[7]], MaxDailyUse[[10]],
                                                  MaxDailyUse[[2]], MaxDailyUse[[5]], MaxDailyUse[[8]], MaxDailyUse[[11]],
                                                  MaxDailyUse[[3]], MaxDailyUse[[6]], MaxDailyUse[[9]], MaxDailyUse[[12]]))
MaxDailyUse[[1]]
```

```
## [1] 34.5000
## 12 Levels: 32.1000 34.4500 34.5000 36.0600 36.5800 36.7300 37.3300 ... 46.6500
```

```
#5
MyLinePlot <- ggplot()+
  geom_line(data=MyFrame, aes(x=Date, y=MaxDailyUse))

MyLinePlot
```



6. Note that the PWSID and the year appear in the web address for the page we scraped. Construct a function with two inputs - “PWSID” and “year” - that:

- Creates a URL pointing to the LWSP for that PWSID for the given year
- Creates a website object and scrapes the data from that object (just as you did above)
- Constructs a dataframe from the scraped data, mostly as you did above, but includes the PWSID and year provided as function inputs in the dataframe.
- Returns the dataframe as the function’s output

#6.

Note: I am building from the structure demonstrated in the lesson file.

```
HarvestSite <- function(the_year, the_facility){
```

```
  #Retrieve the website contents
```

```
  FunctionSite <- read_html(paste0('https://www.ncwater.org/WUDC/app/LWSP/report.php?pwsid=',  
                                   the_facility, '&year=', the_year))
```

```
  #Scrape the data items
```

```
  SystemName <- FunctionSite %>% html_nodes("div+ table tr:nth-child(1) td:nth-child(2)") %>% html_text()
```

```
  PWSID <- FunctionSite %>% html_nodes("td tr:nth-child(1) td:nth-child(5) ") %>% html_text()
```

```
  Ownership <- FunctionSite %>% html_nodes("div+ table tr:nth-child(2) td:nth-child(4)") %>% html_text()
```

```
  MaxDailyUse <- FunctionSite %>% html_nodes("th~ td+ td") %>% html_text()
```

```
  #Convert to a dataframe
```

```
  Withdrawals <- data.frame("Month" = c(1,7,2,8,3,9,4,10,5,11,6,12),
```

```

        "Year" = rep(the_year,12),
        'Ownership' = rep(Ownership,12),
        'PWSID' = rep(PWSID, 12),
        'System' = rep(SystemName, 12),
        'MaxDailyUse' = as.numeric(c(MaxDailyUse[[1]],MaxDailyUse[[4]],MaxDailyUse[[7]],
        MaxDailyUse[[2]],MaxDailyUse[[5]],MaxDailyUse[[8]],MaxDailyUse[[11]],
        MaxDailyUse[[3]],MaxDailyUse[[6]],MaxDailyUse[[9]],MaxDailyUse[[12]]))

mutate(Date = my(paste(Month,"-", Year))) %>%
select(Date,Ownership, PWSID, System, MaxDailyUse)

#Pause for a moment - scraping etiquette
#Sys.sleep(1) #uncomment this if you are doing bulk scraping!

#Return the dataframe
return(Withdrawals)
}

```

7. Use the function above to extract and plot max daily withdrawals for Durham (PWSID='03-32-010') for each month in 2020. Then show the structure of the dataframe with either the `str()` or the `glimpse()` function.

```

#7
DurhamWithdrawals <- HarvestSite(2020,'03-32-010')

str(DurhamWithdrawals)

```

```

## 'data.frame':   12 obs. of  5 variables:
## $ Date          : Date, format: "2020-01-01" "2020-07-01" ...
## $ Ownership     : chr  "Municipality" "Municipality" "Municipality" "Municipality" ...
## $ PWSID         : chr  "03-32-010" "03-32-010" "03-32-010" "03-32-010" ...
## $ System        : chr  "Durham" "Durham" "Durham" "Durham" ...
## $ MaxDailyUse   : num  36 32 37.3 32.4 37 ...

```

8. Use the function above to extract data for Cary (PWSID = '03-92-020') in 2020. Combine this data with the Durham data collected above and create a plot that compares Cary's to Durham's water withdrawals.

```

#8
CaryWithdrawals <- HarvestSite(2020, "03-92-020")

CombinedData <- full_join(x = CaryWithdrawals, y = DurhamWithdrawals)

```

```
## Joining with `by = join_by(Date, Ownership, PWSID, System, MaxDailyUse)`
```

```

CombinedData$System <- as.factor(CombinedData$System)
str(CombinedData)

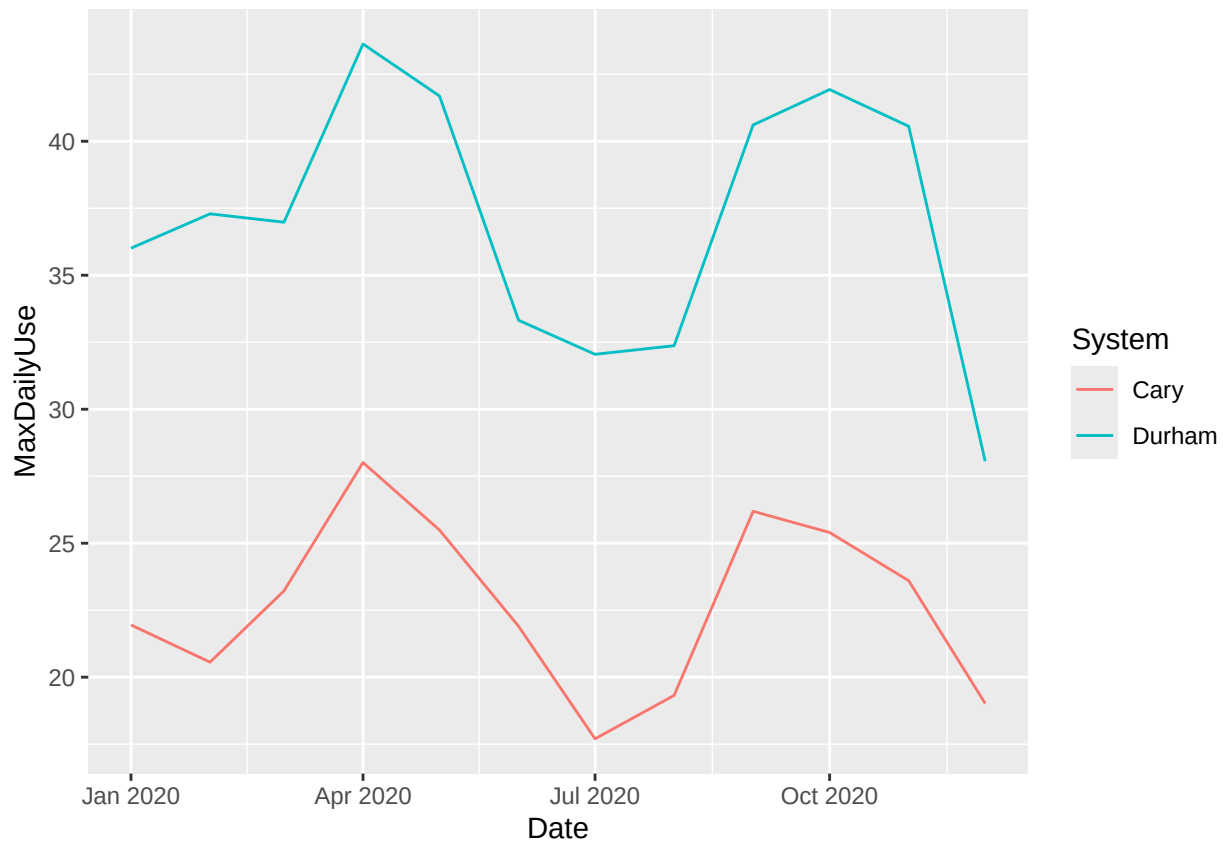
```

```

## 'data.frame':   24 obs. of  5 variables:
## $ Date          : Date, format: "2020-01-01" "2020-07-01" ...
## $ Ownership     : chr  "Municipality" "Municipality" "Municipality" "Municipality" ...
## $ PWSID         : chr  "03-92-020" "03-92-020" "03-92-020" "03-92-020" ...
## $ System        : Factor w/ 2 levels "Cary","Durham": 1 1 1 1 1 1 1 1 1 1 ...
## $ MaxDailyUse   : num  21.9 17.7 20.6 19.3 23.2 ...

```

```
ggplot()+
  geom_line(data=CombinedData, aes(x=Date, y=MaxDailyUse, color= System))
```



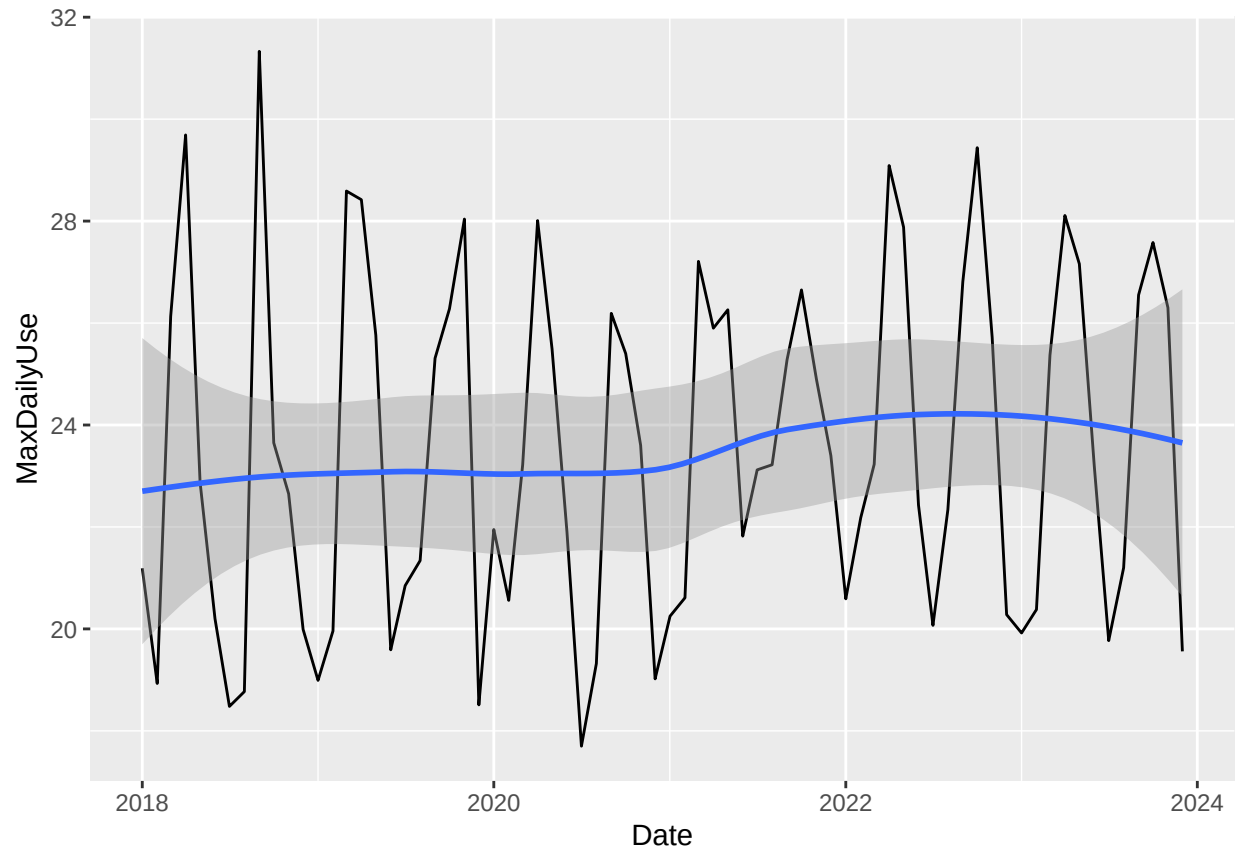
9. Use the code & function you created above to plot Cary’s max daily withdrawal by months for the years 2018 thru 2023. Add a smoothed line to the plot (method = ‘loess’).

TIP: See Section 3.2 in the “10_Data_Scraping.Rmd” where we apply “map2()” to iteratively run a function over two inputs. Pipe the output of the map2() function to bind_rows() to combine the dataframes into a single one, and use that to construct your plot.

```
#9
Years <- c(2018:2023)
Cary6times <- rep("03-92-020",6)
CaryYears <- map2(Years, Cary6times, HarvestSite)
LongCary <- bind_rows(CaryYears)

ggplot(data=LongCary, aes(x=Date, y=MaxDailyUse))+
  geom_line()+
  geom_smooth(method='loess')
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



Question: Just by looking at the plot (i.e. not running statistics), does Cary have a trend in water usage over time? > Answer: Relatively steady trend, with minor increase. >